

Part 1: Title, goal statement, and conventions

Team members: Evan Heidenreich, Nick Rigg, Jacob Curtis

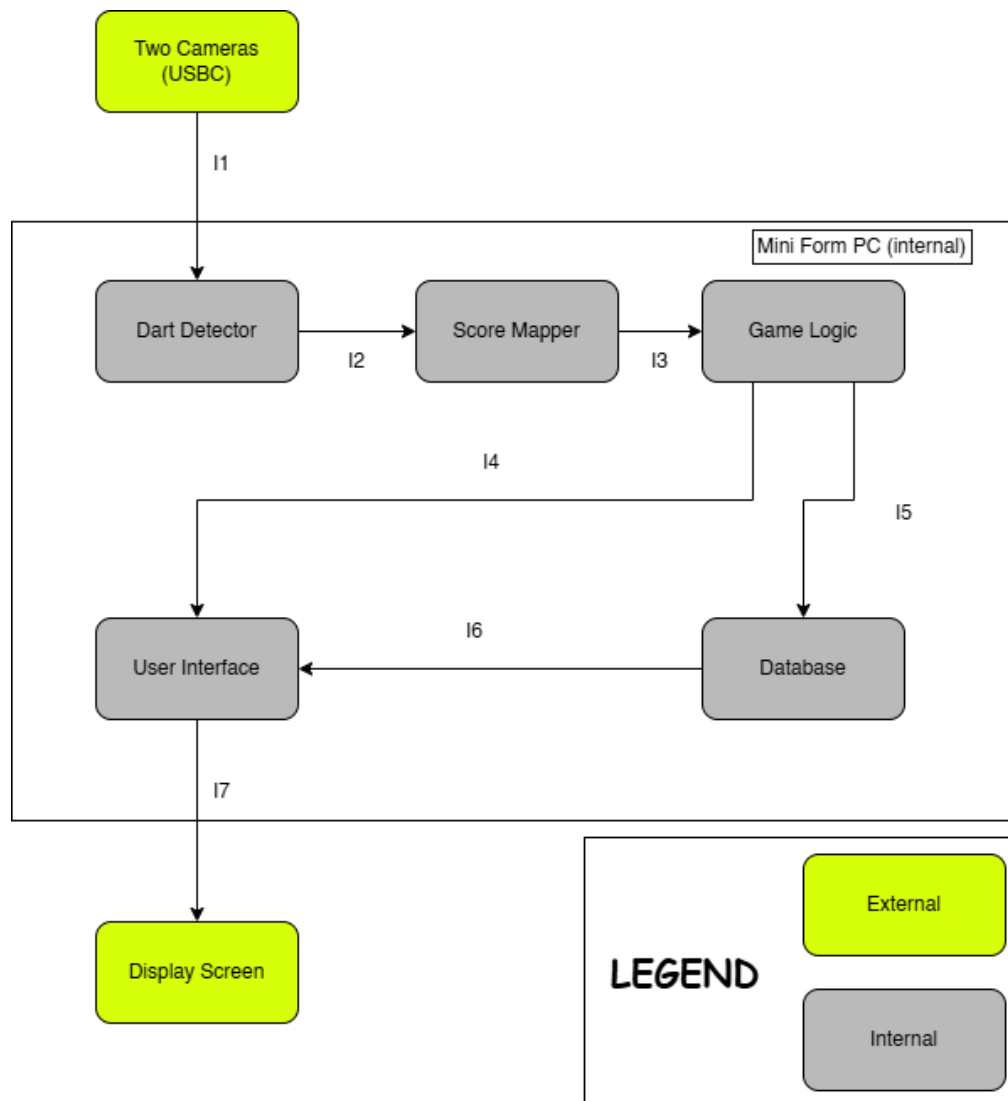
Goal statement: DigiDarts automatically detects where each dart lands on a dartboard using computer vision, scores the game in real time, and tracks player statistics over time.

Basic input: synchronized video frames from two cameras aimed at the board.

Basic output: the score for each dart, updated game state on screen, and saved player stats.

Part 2: Block Diagram

This Diagram showcases a high-level view of how DigiDarts will function. The goal of this block diagram is to show the individual components of DigiDarts.



Part 3: Component responsibility table

Component	Responsibility	In	Out	Owner
Dart Detector	Locate where the dart is on the board	I1	I2	Evan
Score Mapper	Converts where the dart is to the point value	I2	I3	Nick
Game Logic	Applies game rules	I3	I4, I5	Jacob
Database	Stores and retrieves all player stats	I5	I6	Evan
User Interface	Displays score, stats, and other to the player	I4, I6	I7	Nick

Part 4: Interface Specification Table

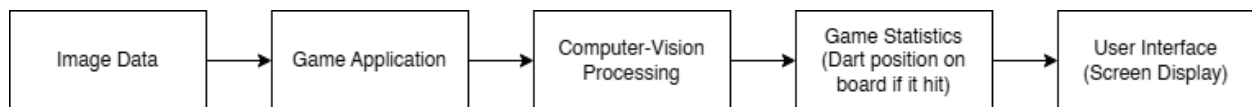
Input	Output	Data Format	Protocol	Error Handling
Cameras	Dart Detector	Video frames	USBC	Camera issues such as disconnection or offline; UI will warn player
Dart Detector	Score mapper	JSON	Function Call	Didn't detect/bad data; Score mapper discards it
Game Logic	Database	Supabase	Cloud base database	Write fails or timeouts; Game logic continues, records are queued locally, retries until succeeds; UI shows "Not Synced" Rejected writes are logged not retried.
Game Logic	User Interface	JSON	API	UI not responding; Game saves state until UI is back running
User Interface	Display Screen	Video signal	HDMI / DisplayPort	No display; Game and UI keep running until display screen turn back on

Below is an example of the interface payload for the I5 Game Logic to Database. We are using JSON formatting to ingest data into Supabase. This payload showcases a singular

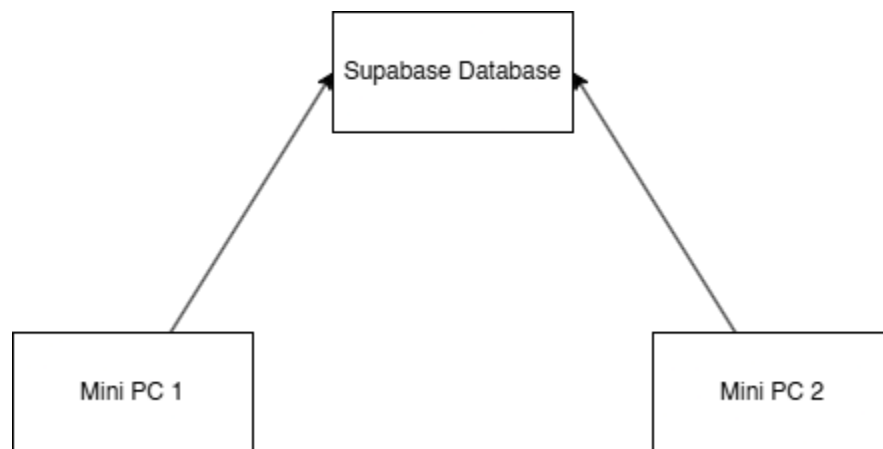
throw during a players turn where we capture a unique individual throw id, unique game id, and unique player id and the corresponding values of that particular throw.

```
{  
  "throw_id": "ijdigj874uguhu4h8qu9u4jchc4if  
  "game_id": "g-20261001-0042",  
  "player_id": "p-0007",  
  "game_type": "501",  
  "turn": 4,  
  "dart": 2,  
  "segment": 20,  
  "multiplier": 3,  
  "points": 60,  
  "remaining": 141,  
  "timestamp": "2026-10-01T21:14:32Z"  
}
```

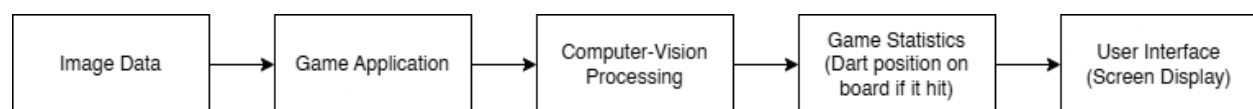
Part 5: Data-Flow Diagram



Part 6: Architecture Pattern and Justification



Possible Architecture Pattern 1: Client-Server Pattern



Possible Architecture Pattern 2: Pipeline / Data-Flow Pattern

We decided to go with Architecture Pattern 2 as this project is less about creating a service with multiple clients that communicate with the same database, and more of a singular client that communicates with a database to keep track of prior high scores for instance. The main bulk of the senior design project surrounds computer vision with regards to where darts land on the board, the game logic, and the user interface.

Part 7: Decision Log

Decision 1: Two cameras instead of one

- **Decision:** Use two cameras aimed at the board from different angles
- **Alternatives considered:** One camera; three or more cameras
- **Why this won:** A single camera won't be enough because it will have to be on the side and won't be accurate. Three or more cameras are unnecessary and cost more.

Decision 2: Ubuntu Linux as the operating system

- **Decision:** Ubuntu Linux on the mini-PC
- **Alternatives considered:** Other operating systems
- **Why this won:** Ubuntu is free and uses fewer background resources than Windows, which leaves more of the mini-PC's limited CPU for video processing. It also has solid support for USB cameras and computer vision tools.

Decision 3: Supabase as the database

- **Decision:** Supabase as a hosted database service
- **Alternatives considered:** A local database on the mini-PC like SQLite or other
- **Why this won:** A hosted database lets stats be viewed from other devices, like a phone, and means no database server to maintain.

Decision 4: Arducam cameras

- **Decision:** Use Arducam USB cameras
- **Alternatives considered:** Standard webcams.
- **Why this won:** Has great resolution, frame rate, or lens field of view, and compare them with the alternative. While it was expensive, we got funding for these cameras.